

模块化与信息隐藏篇期末复习

依据：04-02 面向对象的模块化与信息隐藏课件，123.txt，语音版重点，考试题目类型与重点，2020/2022 往年卷。

使用建议

本篇的最高优先级是类设计原则。做题时先识别代码或类图中的依赖和变化，再回答“违反什么原则、为什么、怎样重构”。控制耦合和印记耦合虽然不是今年最重内容，但往年卷直接考过，仍应会判断。

一、考试定位

优先级	知识点	依据	典型题型
高	类设计原则 P1-P14，尤其 OCP、DIP、SRP、ISP、LSP	答疑明确“类的原则都会考”	给代码/类图，判断符合或违反哪些原则并优化
高	信息隐藏、面向接口、封装变化	语音重点；也是设计模式的基础	解释为什么要抽象接口、怎样隔离变化
中高	控制耦合、印记耦合	123.txt 专门展开；2020/2022 有类似题	判断耦合类型并重构
中	高内聚低耦合、消息链、迪米特法则、组合优于继承	课件基础	分析依赖、消息链和继承层次
参考	包原则与循环依赖	答疑说本次非重点，但 2022 往年题出现	识别循环依赖并通过接口/第三方包消除

一句话定位

模块化与信息隐藏关心的是“变化能否被限制在一个模块内部”。高内聚让一个模块专心做一件事，低耦合让模块之间只通过稳定、最小的接口合作。

二、模块化基础

1. 什么是模块

模块是可以独立设计、实现和测试的自包含软件单元。面向对象系统中可以有方法级、类级和包级模块。模块的边界应围绕职责和变化原因建立，而不是机械地按文件数量划分。

粒度	构件	职责边界
方法级	Method / Function	一个清晰操作
类级	Class	相关状态与行为能力
包级	Package / Namespace	相关类的集合与对外接口

什么是模块 (Module)

A **module** is a self-contained unit of software that can be independently designed, implemented, and tested.

在面向对象中，模块有三种粒度：

粒度	对应构件	职责边界
方法级	Method / Function	单一操作
类级	Class	状态 + 行为
包级	Package / Namespace	相关类集合

模块化的两个核心指标

耦合 (Coupling)

模块之间的依赖强度

→ 越低越好

内聚 (Cohesion)

模块内部元素的关联程度

→ 越高越好

设计目标： 高内聚、低耦合

SWEBOK v4 §1.4 · Modularization | §1.4 · Coupling | §1.4 · Cohesion

6

课件截图：模块化的两个核心指标是耦合和内聚，目标是高内聚、低耦合。

2. 内聚与耦合

- 内聚：模块内部元素之间的关联程度，越高越好。高内聚意味着职责集中、变化原因少。
- 耦合：模块之间的依赖强度，越低越好。低耦合意味着一个模块的修改不容易波及其他模块。
- 高内聚与低耦合相互促进：职责分散常导致大量跨模块调用，职责集中则接口更小。

Cascading Message（消息链）问题

```
// 问题代码：消息链 (Train Wreck)
String city = customer
    .getAddress()
    .getCity()
    .toUpperCase();
```

问题：Customer 的调用方知道了：

- Customer 有 Address
- Address 有 City
- City 是 String

三层实现细节全部暴露！

两种修复方案

方案 A：引入局部变量

```
Address addr = customer.getAddress();
City city = addr.getCity();
String name = city.toUpperCase();
```

可读性提升，但耦合依然存在。

方案 B：委托（推荐）

```
// 在 Customer 中封装
String city = customer.getCityName();
```

调用方只需知道 Customer。

10

课件截图：Cascading Message 问题与委托修复。

3. 控制耦合

控制耦合是调用方传入 flag、类型码或字符串，用它控制被调用方法内部选择哪段逻辑。调用方和被调用方必须共同理解这些控制值的含义。

```
void addCoupon(User user, int type) {
    switch (type) {
        case STUDENT: user.addCoupon(new FiveYuanCoupon()); break;
        case TEACHER: user.addCoupon(new TenYuanCoupon()); break;
    }
}
```

改进方向：把类型差异封装为多态对象，让统一接口负责行为；或把不同职责拆成不同方法。新增类型时增加实现类，而不是修改 switch。

4. 印记耦合 Stamp Coupling

印记耦合是把完整数据结构或对象传给方法，但方法只使用其中少量字段。被调用方被迫依赖它不需要的结构。

```
// ■■■■ employee.email■■■■■ Employee
void sendEmail(Employee employee) {
    mailer.send(employee.getEmail());
}

// ■■■■■■■■■■
void sendEmail(String email) {
    mailer.send(email);
}
```

与 VO 的联系

展示层只应得到界面真正需要的数据。返回 VO 而不是完整 PO，也是在减少不必要的数据依赖和印记耦合。

5. 控制耦合与印记耦合对比

类型	识别特征	问题	常见修复
控制耦合	flag/type 控制内部 if/switch	双方共同依赖控制码语义	多态、策略、拆分方法
印记耦合	传整个对象，只用少数字段	依赖超出实际需要的数据结构	只传必要值、窄接口、专用 DTO/VO

四、面向接口与接口隔离

1. 面向接口编程

```
// ■■■■■■
ArrayList<String> names = new ArrayList<>();

// ■■■■■■
List<String> names = new ArrayList<>();
```

- 调用者依赖接口契约，而不是某个具体实现。
- 实现类可以替换，调用代码通常不变。
- 支持多态、依赖注入和 Mock 测试。

P4 · 面向接口编程

Program to an interface, not an implementation.

```
// 不好：依赖具体类
ArrayList<String> list = new ArrayList<>();

// 好：依赖接口
List<String> list = new ArrayList<>();
```

好处：

- 调用方只依赖接口契约，不依赖实现
- 切换实现类无需修改调用方
- 支持多态、Mock 测试

面向接口编程将访问耦合从参数变量级降低到仅依赖接口，是降低耦合的核心手段。

SWEBOK v4 §1.4 · Separation of Interface and Implementation | §5.4 · Object-Oriented Design (SOLID)

12

课件截图：面向接口编程是降低耦合的核心手段。

2. 接口隔离原则 ISP

客户端不应被迫依赖它不使用的方法。肥接口会迫使实现类提供无意义方法，甚至抛出 `UnsupportedOperationException`。应按客户端角色拆成更小、更专一的接口。

P6 · 接口隔离原则（ISP）

Clients should not be forced to depend on interfaces they do not use.

肥接口问题

```
interface Worker {
    void work();
    void eat();    // Robot 不需要!
    void sleep(); // Robot 不需要!
}

class Robot implements Worker {
    public void work() { ... }
    public void eat() { throw new UnsupportedOperationException(); }
    public void sleep(){ throw new UnsupportedOperationException(); }
}
```

ISP 重构

```
interface Workable {
    void work();
}

interface Feedable {
    void eat();
    void sleep();
}

class Robot implements Workable { ... }
class Human implements Workable,
    Feedable { ... }
```

接口细化后，`Robot` 不再被迫实现无关方法，调用方只依赖自己真正需要的接口。

SWEBOK v4 §5.4 · Object-Oriented Design (SOLID — Interface Segregation) | §1.4 · Separation of Interface and Implementation

13

课件截图：将 `Worker` 拆成 `Workable` 与 `Feedable`，使 `Robot` 不依赖无关方法。

五、继承耦合与里氏替换

1. 里氏替换原则 LSP

子类型必须能够替换父类型而不改变程序正确性。判断 IS-A 不能只看自然语言或数学集合，更要看行为契约。

契约要素	LSP 要求
前置条件	子类不能比父类更严格
后置条件	子类不能比父类保证更少
不变式	子类必须维护父类的全部不变式
异常/语义	不能把正常行为改成异常或完全不同的含义

P7 · 里氏替换原则 (LSP)

子类型必须能够替换其父类型而不改变程序的正确性。
— Barbara Liskov, 1987

经典反例：正方形 ≠ 长方形

```
class Rectangle {  
    void setWidth(int w) { ... }  
    void setHeight(int h) { ... }  
    // 不变式: w 与 h 独立  
}  
  
class Square extends Rectangle {  
    void setWidth(int w) {  
        super.setWidth(w);  
        super.setHeight(w); // 破坏不变式!  
    }  
}
```

为什么违反 LSP?

```
void resize(Rectangle r) {  
    r.setWidth(5);  
    r.setHeight(3);  
    // 期望面积 = 15  
    assert r.area() == 15; // Square 中失败!  
}
```

Square 在几何上是 **Rectangle** 的子集，但在行为契约上不能替换。"IS-A" 关系应由行为定义，而非数学定义。

 SWEBOOK v4 §5.4 · Object-Oriented Design (SOLID — Liskov Substitution)

16

课件截图：正方形继承长方形会破坏宽高独立的不变式，是经典 LSP 反例。

2. 组合优于继承

- 继承是白盒复用，子类依赖父类实现细节；父类修改可能破坏所有子类。
- 组合是黑盒复用，只依赖对象接口，运行时还可以动态替换。
- 只有真正稳定的行为 IS-A 关系才适合继承；行为变化维度多时优先组合。

P8 · 组合优于继承

Favor object composition over class inheritance.

— GoF, *Design Patterns*, 1994

继承的问题

- 白盒复用：子类依赖父类实现细节
- 父类修改可能破坏所有子类
- 继承层次难以扩展（多个维度变化时）
- 强迫遵守 IS-A 语义

组合的优势

- 黑盒复用：只依赖接口
- 运行时可动态切换组合对象
- 职责分离更清晰
- 无需父类 IS-A 约束

```
// 继承: Stack extends Vector (历史错误)
// 组合: Stack 内部持有 Vector
class Stack<T> {
    private Vector<T> storage = new Vector<>();
    public void push(T item) { storage.add(item); }
    public T pop() { return storage.remove(...); }
}
```

课件截图：组合通过接口复用行为，职责和变化边界更清晰。

六、信息隐藏与封装

1. 信息隐藏的本质

Parnas 的核心思想是：每个模块都应隐藏一个重要设计决策。隐藏的不只是字段，而是实现细节和未来变化。变化被限制在模块内部，其他模块只依赖稳定接口。

秘密	含义	示例
主要秘密	模块职责可能怎样改变	折扣规则、持久化职责是否变化
次要秘密	模块内部如何实现可能改变	List 改为 Set、MySQL 改为 SQL Server

信息隐藏的本质

每个模块应该对其他模块隐藏一个重要的设计决策。
— David Parnas, 1972

两类秘密 (Secrets)

主要秘密 (Primary Secret)

= 职责变更的可能性

→ 这个类应该做什么会变化吗?

次要秘密 (Secondary Secret)

= 实现变更的可能性

→ 这个类如何做会变化吗?

为什么要隐藏?

变化是软件的常态。

将变化的可能性封装在模块内部，使得修改只影响该模块，而不波及其他模块。

信息隐藏 = 隔离变化 = 降低耦合

SWEBOK v4 §1.4 · Encapsulation (information hiding) | §1.4 · Modularization

21

课件截图：信息隐藏就是隔离变化、降低耦合。

2. 封装的五种类型

类型	隐藏内容	核心手段
A 数据类型	内部数据表示	private 字段和受控访问器
B 内部结构	集合、数组的组织方式	不直接暴露内部容器
C 其他对象引用	内部持有哪些对象	不返回可修改的内部引用
D 类型信息	具体运行时类型	面向接口、多态调度
E 潜在变更	可能变化的算法或策略	策略、工厂、依赖注入

封装的五种类型（A—E）

类型	隐藏内容	核心手段
A — 数据类型（ADT）	内部数据表示	访问器/修改器替代 public 字段
B — 内部结构	数据的组织结构	不暴露内部容器/数组
C — 其他对象引用	内部持有哪些对象	不直接返回内部对象引用
D — 类型信息（LSP）	对象的具体运行时类型	面向接口，多态调度
E — 潜在变更	可能变化的实现策略	策略模式、依赖注入

类型 A—C 处理当前实现细节的封装；类型 D—E 处理未来变化的封装。这五种类型覆盖了面向对象封装的完整语义。

SWEBOK v4 §1.4 · Encapsulation | §1.4 · Separation of Interface and Implementation | §1.4 · Abstraction

24

课件截图：封装 A-E 覆盖当前实现细节和未来变化。

3. 封装潜在变更

```
interface SortStrategy {
    void sort(int[] values);
}

class Sorter {
    private final SortStrategy strategy;
    Sorter(SortStrategy strategy) { this.strategy = strategy; }
    void sort(int[] values) { strategy.sort(values); }
}
```

Sorter 不知道冒泡排序还是快速排序，只依赖 SortStrategy。算法改变时替换实现对象，稳定流程不变。这同时体现信息隐藏、组合、OCP 和 DIP。

类型 E：封装潜在变更

将可能变化的部分隔离为独立模块，其余部分依赖稳定的接口。

问题：变化硬编码在类内部

```
class Sorter {
    void sort(int[] arr) {
        // 直接写死冒泡排序
        // 若要换快速排序，必须修改本类
        bubbleSort(arr);
    }
}
```

策略模式封装变更

```
interface SortStrategy {
    void sort(int[] arr);
}

class Sorter {
    private SortStrategy strategy;

    Sorter(SortStrategy s) {
        this.strategy = s;
    }

    void sort(int[] arr) {
        strategy.sort(arr); // 委托给策略
    }
}
```

封装类型 E 与**开闭原则（OCP）**直接对应：对扩展开放，对修改关闭。

29

课件截图：类型 E 将算法变化隔离到策略实现中。

七、类和方法设计原则

1. 开闭原则 OCP

软件实体应对扩展开放、对修改关闭。新增变化类型时优先增加实现类，而不是修改稳定类中的 if/switch。OCP 不是禁止一切修改，而是建立稳定抽象，使常见变化通过扩展完成。

P11 · 开闭原则 (OCP)

Software entities should be open for extension, but closed for modification.

— Bertrand Meyer

违反 OCP 的典型模式

```
// 每次增加新类型都要修改
class DiscountCalculator {
    double calculate(Order order) {
        if (order.type == VIPORDER)
            return order.total * 0.8;
        else if (order.type == NEWUSER)
            return order.total * 0.9;
        // 增加新类型需修改本类...
    }
}
```

符合 OCP 的重构

```
interface DiscountPolicy {
    double apply(double total);
}

class VipDiscount implements DiscountPolicy {
    public double apply(double t) {
        return t * 0.8;
    }
}

class DiscountCalculator {
    private DiscountPolicy policy;
    // 新增类型只需新增实现类
    double calculate(Order o) {
        return policy.apply(o.getTotal());
    }
}
```

33

课件截图：折扣策略通过接口和多态实现 OCP。

2. 依赖倒置原则 DIP

高层模块不依赖低层具体实现，两者都依赖抽象；细节反过来实现高层定义的接口。常见实现手段是接口、构造器注入和工厂。

P12 · 依赖倒置原则 (DIP)

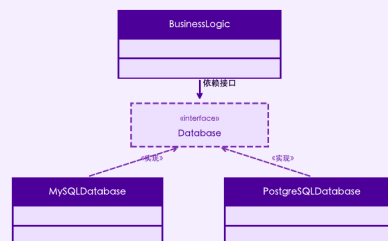
高层模块不应依赖低层模块；两者都应依赖抽象。
抽象不应依赖细节；细节应依赖抽象。

违反 DIP



```
class BusinessLogic {
    private MySQLDatabase db; // 依赖具体类!
    void process() {
        db.query(...);
    }
}
```

符合 DIP



```
class BusinessLogic {
    private Database db; // 依赖接口
    BusinessLogic(Database db) {
        this.db = db; // 依赖注入
    }
}
```

34

课件截图：业务逻辑依赖 Database 接口，而不是 MySQLDatabase。

3. 单一职责原则 SRP

一个类应该只有一个变化原因。把薪资计算、数据库保存、报表生成放入 Employee，会受到 HR、IT、财务三类变化影响，应拆为不同职责对象。

P13 · 单一职责原则 (SRP)

A class should have only one reason to change.
— Robert C. Martin

违反 SRP

```
class Employee {  
    String name;  
    double salary;  
    // 职责1: 薪资计算 (HR 规则)  
    double calculatePay() { ... }  
    // 职责2: 数据库持久化 (IT 基础设施)  
    void saveToDatabase() { ... }  
    // 职责3: 报表生成 (财务需求)  
    String generateReport() { ... }  
}
```

三个不同的"变化原因"混在一个类中。

SRP 重构

```
class Employee { // 只管员工数据  
    String name;  
    double salary;  
}  
  
class PayCalculator { // 只管薪资计算  
    double calculate(Employee e) { ... }  
}  
  
class EmployeeRepository { // 只管持久化  
    void save(Employee e) { ... }  
}  
  
class EmployeeReport { // 只管报表  
    String generate(Employee e) { ... }  
}
```

SRP 与信息隐藏直接关联：每个类只隐藏一个"变化原因" (= Parnas 的"设计决策")。SRP 违反的症状是"God Class"——一个类知道太多、做太多。

35

课件截图：SRP 把不同变化原因拆到独立类中。

4. SOFA 方法级原则

字母	原则	判断方法
S	Short	方法保持短小，过长则提取辅助方法
O	One thing	方法只做一件事，名称不需要用 and 描述
F	Few arguments	参数尽量少，避免调用者知道过多细节
A	Abstraction level consistency	同一方法保持相同抽象层次

P14 · SOFA：方法级设计原则

SOFA = Short · One thing · Few arguments · Abstraction level consistency
— 面向方法粒度的设计原则 (与 SOLID 的类粒度原则互补)

S — Short (方法要短)

- 方法体通常 ≤ 20–30 行
- 过长 → 分解为命名清晰的辅助方法

O — One thing (只做一件事)

- 方法名能用一个短句完整描述其行为
- 若需要"and"才能描述 → 考虑拆分

F — Few arguments (参数要少)

- 参数越多，调用方知道的细节越多 → 耦合越高
- 超过 3 个参数时考虑引入参数对象

A — Abstraction level consistency (抽象层次一致)

```
// 违反 A: 高层业务逻辑混入底层实现  
void processOrder(Order o) {  
    validateOrder(o); // 高层  
    db.execute("INSERT ..."); // 底层!  
}  
  
// 修复: 把底层细节封装为方法  
void processOrder(Order o) {  
    validateOrder(o);  
    persistOrder(o); // 统一在高层  
}
```

36

课件截图：SOFA 从方法粒度约束长度、职责、参数和抽象层次。

5. P1-P14 总表

编号	原则	核心判断
P1	全局变量有害	是否存在穿透模块的隐式共享状态
P2	优先显式表达	约束是否依赖隐含约定
P3	DRY	同一知识或决策是否重复编码
P4	面向接口	调用方是否依赖具体实现
P5	迪米特法则	是否通过消息链访问间接对象
P6	ISP	客户端是否被迫依赖无关方法
P7	LSP	子类能否无损替换父类
P8	组合优于继承	能否用黑盒组合降低继承耦合
P9	封装变化	可变部分是否与稳定部分隔离
P10	权限最小化	成员是否从 private 开始按需开放
P11	OCP	新增类型是否总要修改旧代码
P12	DIP	高层是否依赖抽象而非低层细节
P13	SRP	一个类是否有多个变化原因
P14	SOFA	方法是否短、小职责、少参数、同抽象层

八、往年题例题与答案

例题 1：2020 耦合判断

```
int valid_month(Date date) {  
    return date.month >= 1 && date.month <= 12;  
}  
  
int valid(String value, int type) {  
    switch (type) {  
        case STRING: ...  
        case DATE: ...  
    }  
}
```

问法	参考答案
validate_request 与 valid_month 是什么耦合？	印记耦合。valid_month 接收完整 Date，却只使用 month。改为 validMonth(int month)，只传必要数据。
validate_request 与 valid 是什么耦合？	控制耦合。调用者传 type 控制 valid 内部 switch，双方都依赖类型码语义。改为 validString、validDate 等专一方法，或定义 Validator 接口并用多态实现。

例题 2：2022 循环依赖

题目：用户包持有优惠券包中的优惠券，优惠券包又查询用户包信息以发放优惠券。

问题	参考答案
违反什么原则？	形成 UserPackage -> CouponPackage -> UserPackage 的循环依赖，违反无环依赖原则，也使两个包无法独立修改、测试和部署。
如何修改？	把发券用例协调职责放入独立 Application/Service 包；Coupon 只表达优惠券规则，User 只表达用户状态。或在稳定一侧定义接口，通过 DIP 让另一侧实现，打断反向具体依赖。
类图方向	IssueCouponService 依赖 UserRepository 与 CouponPolicy 接口；User 和具体 Coupon 不再双向查询。依赖箭头都指向抽象接口。

当前考试权重

答疑明确包原则不是本次重点，但往年卷出现过，所以应会识别循环依赖。真正高优先级仍是类设计原则和基于代码的判断。

例题 3：发券代码综合分析

现象	原则分析	改进
type + switch 选择优惠券	控制耦合；违反 OCP；调用者和方法共享类型码	定义 CouponPolicy/Issuer 接口，用不同实现封装发券行为
方法直接 new 具体 Coupon	高层依赖具体类，违反 DIP	通过工厂或依赖注入获得策略
新增用户类型必须改 switch	稳定类对修改开放，变化未封装	新增实现类并注册，不修改核心流程
多个分支重复 user.addCoupon	重复表达相同操作，违反 DRY	先得到 Coupon，再统一调用一次 addCoupon

九、答题模板与速记

1. 原则分析题模板

- 1 指出代码或类图中的具体依赖、分支、字段暴露或职责混合。
- 2 写出违反/体现的原则名称。
- 3 解释变化会如何传播，或为何增加耦合、破坏替换。
- 4 给出重构：拆职责、提接口、依赖注入、多态、委托、缩小参数。
- 5 说明重构后新增需求需要增加什么、哪些稳定代码不必修改。

2. 高频速记

看到什么	优先想到
flag/type + switch	控制耦合、OCP、多态/策略
传整个对象只用一项	印记耦合、最小接口
a.getB().getC()	消息链、迪米特法则、委托
public 字段/集合	封装缺失、权限最小化、受控方法
子类抛不支持异常	LSP/ISP 违反，拆接口
业务类直接 new 数据库实现	DIP 违反，接口 + 注入
一个类又计算又保存又报表	SRP 违反，按变化原因拆分
新增类型总改 if/switch	OCP 违反，封装变化并用多态

最后检查

答题不要只报原则名称。至少写出“代码证据 - 原则 - 影响 - 重构 - 扩展效果”五个环节。